scientific reports

Check for updates

OPEN

Detection and analysis of prompt injection in indian multilingual large language models

Jagadeesan Srinivasan[1]✉, Silla Ann Regi[2], Anbarasa Kumar Anbarasan[3], Suresh A[4], Vetriselvi T[5] & Sivakumar Venu[6]✉

With the use of Large Language Models (LLMs) becoming more significant in many applications, it has become a great need to ask the question how these LLMs can be made safe and robust, and hence make it behave in an ethical way. Among the most seen and evident vulnerabilities or loopholes in LLM are the attacks termed as prompts injections, which is a key vulnerability. Even though this issue is being extensively studied, most of the available and existing literature and defense systems are mostly focussed on English language prompts. As of now, there are no publicly available datasets or any known defense mechanisms that works with the aim of detecting and/or preventing the so-called prompt injection in Indian regional or code-mixed languages associated. The project aims to identify or propose a mechanism to find an answer on exactly how to overcome this significant research gap by assessing the reaction of various LLMs to the injection-style prompts typed in Indian languages particularly Hindi and its code-mixed version, Hinglish. A clean dataset of size 4000 prompts including safe and malicious prompts in Hindi and Hinglish (Hindi-2000 prompts and Hinglish-2000 prompts) is made and a hybrid classifier is proposed that finds to detect injection attempts using their language patterns and other intents or features. Various algorithms including Transformer, Distil Transformer, SVM, XGBoost, Random Forest and Logistic Regression are trained and tested. The integration of a rule-based layer and a transformer model was shown to perform better than all baselines, with 99.70% accuracy and high precision (99.65%), recall (99.75%), and F1-score (99.70%), which demonstrates that its detection is very reliable. The system provides a new direction in ensuring the security of LLM interfaces against prompt-based injection attacks, especially in a diverse and multilingual user population like that in India.

Keywords Prompt injection, Large language models, Multilingual NLP, Machine learning, LLM safety, AI prompt vulnerability

With the development of Large Language Models (LLMs) like ChatGPT, Gemini, Claude etc., user's way of interacting with technology have changed. Evolving technology has made it easier for the users to interact with LLMs. These models make it easier in a way that the users are easily able to understand the context and communication. It generates texts as outputs that always resembles human writing. As a result of which, these LLMs have started to be used in various applications in real world including education, research, customer support, software development and many more. It has become an integral part of the normal life of a human who are into technologies, innovations and is aware of the benefits that LLMs bring.

When talking of advantages that LLMs bring, there is an equal number of concerns that could be raised. The security, reliability and ethical use of these LLMs can be one among the major areas where there can be chances that attacks may happen. In an ever-growing world like this, attackers evolve and hence the number of

[1]Department of Software and Systems Engineering, School of Computer Science Engineering and Information Systems (SCORE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. [2]Department of Computer Applications, School of Computer Science Engineering and Information Systems (SCORE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. [3]Department of Information Technology, School of Computer Science Engineering and Information Systems (SCORE), Vellore Institute of Technology (VIT), Vellore, India. [4]Department of Computational Intelligence, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. [5]Department of IoT, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. [6]Department of Analytics, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. ✉email: s.jagadeesan@vit.ac.in; sivakumar.v@vit.ac.in

attacks as well as its intensity of damage increases. For example, LLMs might become immune to manipulations. One among the many numbers of attacks present, prompt injection attacks bring up a serious issue of manipulation. Prompt Injection Attack is a type of attack wherein the attacker or the user tries to override the LLM's internal safeguards and instructions by intentionally crafting malicious input texts to the models. This can lead to exploiting the model to generate harmful or misleading texts as output and can be against the ethical considerations or boundaries. For example, an attacker can try to trick the model and ask it to reveal any of its internal system information or any kind of data that the model is actually prohibited from showing.

Most studies that have dealt with prompt injection attacks focus exclusively on inputs in English language. This could be because English is considered a common medium of communication and could be applicable to a large number of audiences. But this brings up a significant gap when LLMs are made to deploy in multilingual contexts. For example, in a country like India where there is a wide variety of languages varying from state to state and region to region, there are high chances of security concerns when dealing with LLMs. These languages tend to be informal and transliterated. In such cases, English based safety and security mechanisms would fail to interpret the true essence or intent of prompts, hence leaving a large amount of people, systems and data to be vulnerable. Existing work solely focuses on English and there are limited code mixed datasets.

This research aims to build a start to the concept of prompt injection attacks using multilingual languages inputs (focusing mainly on Hindi and Hinglish) to LLMs. This could help bridge the research gap by designing a multilingual detection framework and analysing prompt injection attempts in Indian languages like Hindi and Hinglish.

Contributions of this paper are:

- A manually crafted dataset containing 2000 Hindi and 2000 Hinglish prompts equally divided as safe and injection.
- A hybrid prompt injection detection framework that integrates rule based logic and transformer model.
- An evaluation using both classification and attack metrics.

The rest of the paper is structured in a way which gives a clear and logical presentation of the study. In section "Related Work", a literature review of the related works is presented. Section "Threats to Validity" is devoted to the possible threats to validity that can influence the results of the study. Section "Research Gap" that outlines the research gap and justifies the significance of the suggested method. Section "Methodology" describes the general methodology that shall be used in this work, whereas the proposed system is described in Section "Proposed System". Section "Implementation" outlines implementation aspects and experimental setup of the evaluation of the system. Section "Conclusion" finalizes the paper with a conclusion on the key findings and future research directions.

Related work

The literature review on prompt injection have existed before and a consistent growth in the growth of defenses and security has been clearly found. But the works still remain compounded to English languages and some other foreign languages including Spanish. There exists a broad range of techniques already on English inputs which range from simple ML models on lexical features to graph-based detection as well. Yet there are less particular detection methods found to be inspecting on non-English inputs or code-mixed languages. The review that follows below shows the prior works on prompt injection detection focusing on the datasets used, the methods developed, frameworks, and gaps identified. Previous works as referred has shown a significant growth in addressing the issue of prompt injection attacks in LLMs.

One significant contribution is by Omri, Abdelkar and Hamdi[1] by introducing a specialized dataset called MalPID for performing prompt injection detection in LLM-based applications. They made use of ALBERT base v2 and Logistic Regression classifiers to effectively distinguish between safe normal prompts and injected prompts. They were able to achieve F1-scores of 99% and 97.49% respectively. They have also pointed out how important ethical considerations are and also emphasized the need for securing dataset usage when it comes to AI-driven applications.

Also, in other research done by Abbasi et al.[2] dealt with multilingual prompt injection attacks and evaluated nineteen pre-trained models which was spread across English and Spanish datasets. They found out that stringer robustness was achieved by adversarial fine-tuned multilingual models when compared to general architectures and frameworks. Through this they brought up the need for adaptive defenses cross lingual testing. There are also researchers that have used many machine learning and deep learning models to perform prompt injection detection and have done a comparison among them.

Nair et al[3] had made use of Random Forest and Bidirectional LSTM models followed by use of SHAP-based explainability to enhance the trustworthiness in detection decisions. Random Forest achieved an accuracy of 88.75% and Bi-LSTM achieved 92.5% accuracy. This shows the superiority of sequential models when it comes to contextual text analysis. Jain et al.[4] extended this research by adding more ML and DL models such as Logistic Regression, SVM, Gradient Boosting, CNN, RNN and Bi-LSTM. RNN was found to have the highest detection rate of 94.74%, again showing the importance of deep sequential architectures and frameworks for capturing contextual dependencies.

Researchers have also come up with various hybrid and ensemble techniques to improve robustness of models. Chisty et al.[5] proposed an ensembling of BERT and LSTM built on Glove embeddings showing an accuracy of 94.82%. This approach included both semantic and sequential representations and also outperformed single-model baselines. Also, Rahman et al.[6] made use of pre-trained Multilingual BERT and DistilBERT embeddings along with traditional ML models like Naïve Bayes, Random Forest, SVM and Logistic Regression. This

combination made achieve an accuracy of 96.55%. Both the above-mentioned study researches show that use of fine-tuning large multilingual transformers can further enhance accuracy and generalization.

There are also several studies that focus on advanced stack strategies. Kwon et al.[7] proposed a graph-based prompt injection technique where replacement of sensitive words is done. They are replaced by mathematical expressions that can effectively hide malicious intent. This method has made it achieve a higher success rate when comparing to the existing injection approaches. In another related study by Hackett et al.[8] they have studied the robustness of commercial LLM guardrail systems like Azure Prompt Shield and have depicted that character injection methods have the ability to achieve even up to 100% of evasion success. They have also suggested the development of more comprehensive evaluations that should spread across white-box and black-box systems so that it identifies underlying weaknesses in LLM defenses.

There are also trends that include adversarial attacks which require more defensive frameworks. Hung et al.[9] introduced the concept called Attention Tracker which is a detection method that has the ability to monitor attention distributions within LLMs and check for any abnormal patterns that could indicate injected prompts. The developed system proved to be outperforming all the existing baselines and was seen to be efficient in smaller LLMs also. This had the potential for real-time deployment. Similarly, Lin et al.[10] developed another concept called UniGuardian, which was a unified detection framework that could simultaneously handle prompt injections, backdoor and adversarial attacks. They got an AUROC value of 0.99 that clearly shows great generalization among multiple datasets. They have also pointed out the need to expand the research to non-English languages as well and also reduce the false positives. Systems level security viewpoint was used to explore comprehensive defense frameworks.

Raj et al.[11] had analyzed LLM vulnerabilities that included prompt injection, data poisoning and model inversion and also came up with a multi-layer defense mechanism using dynamic trust scoring. They were able to achieve an accuracy of 96% accompanied with less than 1.8% false positives. Similarly, Joseph et al.[12] made efforts to analyze multiple attack vectors including prompt injection, SQL/XSS exploitation and data leaks. They found out a failure rate of 30% of existing systems when tested with novel injection variants. Hence, the requirement for adaptive countermeasures were emphasized. Another research by Gosmar et al.[13] also brought up a multi-agent NLP framework that had the ability to automatically sanitize outputs and enforce content compliance. These studies prove the effectiveness of mitigating prompt injection and also depict the potential for integrating into autonomous AI pipelines.

There are also several studies that focus on domain-specific or applied contexts. Lan et al.[14] developed a multi-faceted detection system that used fine-tuned BERT model aiming LLM integrated applications. The system achieved over 98% accuracy and demonstrated real time response potential. They also brought into consideration the challenges they faced in detecting indirect prompt injections. Ergan and Onan[15] made use of a classification-driven approach that had the ability to analyze prompt texts as well as response features. They came into a conclusion that the CNN-LSTM cascade model had the best performance with an accuracy of 97% on prompt features. They also suggested that contextual embeddings and real time capabilities could be integrated and hence enhance robustness among various multimodal data types.

Recent works highlights the vulnerability of deep learning-based detection systems to adversarial manipulation and also the significance of robustness oriented training strategies. K. Shaukat et al.[16] came up with a framework that could be used for strengthening neural malware detectors against evasion attacks using adversarial training with multiple attack generation techniques. The study shows that carefully crafted perturbations can lead to classifiers misclassifying the malicious samples and also shows that using adversarially generated samples during training can help in improving resilience to such attacks. Their experimental evaluation across many neural detectors and attack methods shows that robustness can be increased by exposing the models to diverse adversarial scenarios and reinforcing the need to consider adversarial threat models while designing secure intelligent detection systems.

Recent studies by Chen L et al.[17] have explored collaborative multi-agent frameworks that were built on LLMs in order to improve the reasoning reliability and mitigate hallucinations. A multi LLM architecture has been proposed in which there are evaluator and supervisor agents cooperate using iterative refinement and debate in order to extract user value representations from historical interactions. This approach helps demonstrate how modern LLM systems would increasingly depend on coordinated agent roles instead of isolated model inference thereby highlighting the growing complexity of LLM interaction pipelines. Such multi agent environments lead to expansion of potential attack surface for malicious inputs. This reinforces the need of input level safeguarding mechanisms for prompt injection detection.

In addition, there are recent works by Zhong Z et al.[18] that deal with deployment-oriented language model frameworks to investigate collaborative cloud device frameworks for personalized model adaptation. There has been an introduction of a cloud assisted data augmentation strategy which has a server side LLM that generates synthetic data to enhance the training of smaller on device models thereby helping in efficient personalized inference without continuous cloud dependence. These studies show that there is an increase in distributed and real world deployment of LLM systems in which the inputs from the user comes from heterogenous and uncontrolled environments. Such types of deployment scenarios point out the importance of robust input validation and filtering mechanisms so that the reliability and safety of downstream language model processing can be ensured.

Literature surveys have also discussed on the understanding of security challenges that surrounds LLMs and their integration into autonomous systems. Studies carried out on AI agent security shows that the agents that are operating on top of these language models may cause vulnerabilities coming from unpredictable inputs from the users, complex internal reasoning chains, environmental variability and interactions with untrusted entities, wherein these prompt injection attacks and jailbreak attacks can compromise perception and decision making pipelines as discussed by Deng et al.[19]. Xu et al.[20] shows that LLM based agents have the ability to carrying out

multi stage cyber attacks autonomously, lowering the barrier for adversarial activities and amplifying existing system vulnerabilities using automated reasoning and action modules. This emphasizes the requirement for proactive input filtering and defensive monitoring mechanisms. In parallel, there are also surveys by Deng et al.[21] that tells the emergence of model ecosystems such as DeepSeek. This highlights the rapid evolution and widespread deployment of modern LLMs across diverse domains hence giving rise to safety, privacy and ethical concerns that are connected with large scale user interaction and open accessibility. Collectively, all the discussed perspectives point out the need for robust prompt level safeguarding mechanisms, mostly in multilingual environments where variability in inputs bring sup additional complexity, motivating the detection focussed framework that is focussed in this study.

Multilingual safety is always a critical concern when dealing with LLMs. Wang et al.[22] introduced the first large scale multilingual benchmark called XSAFETY that could be used across various LLMs including ChatGPT, PaLM-2, LLaMA-2-Chat and Vicuna models. The study shows that LLMs show significantly lower safety performance when it comes to dealing with non-English languages. They have also added that cross-lingual prompting techniques will be able to reduce unsafe responses up to 42%. They suggested to expand the study and research across bias mitigation, copyright safety and cross-lingual generalization.

Overall, all the above-mentioned studies demonstrate the growth in prompt injection attacks and its detection but also points out the challenges like dataset diversity, multilingual robustness and real time deployability still existing that has to be sorted out. Below Table 1 compares machine learning approach for based Prompt Injection.

Threats to validity

Many of the prompts were translated and transliterated from English to Hindi and Hinglish respectively. This can bring up translation style patterns that the models may learn instead of the real attack behaviour. Manual prompts were also added to the dataset and human verification was also properly done. The dataset can be extended in a way that includes the typical speaking way of local languages. Since the dataset also includes only Hindi and Hinglish languages, extension to other Indian languages is suggested but the results obtained here cannot be generalized since the rule based layer is particularly crafted for Hindi and Hinglish only. Training and Testing were performed on the same dataset with an 80:20 split. Cross dataset or cross platform evaluation was not carried out. Thus, the performance may not be able to completely capture generalization to unseen attack styles. There is a hybrid model that includes a rule based layer that has predefined keywords. This could favour the detection of known attack templates while being less effective for highly obfuscated or semantic prompt injections. However, the transformer model used helps to mitigate this limitation by providing the contextual understanding. The dataset made mainly focusses on direct and template based prompt injection attacks whereas indirect or semantic prompts were not extensively evaluated which makes it a future work to extend the dataset to include more complex prompts and adaptive strategies. LLMs keep on evolving and rapidly updating their safety mechanisms over time. Hence there could be chances that the proposed system's effectiveness may vary over different LLM versions and deployment settings.

Research gap

Though Large Language Models (LLMs) have shown to excel at parsing and generating natural language, they can still be affected by prompt injection attacks (i.e., especially crafted inputs that have side-stepped safety guards and produce malicious or unintended text) and can be used to maliciously design harm to the model.

- Existing literature and solutions that are used to mitigate these vulnerabilities largely assess and treat them on English speaking environments.
- In a country like India where multiple languages are used apart from English, conversely, users address AI systems on a regular basis using regional languages (e.g., Hindi, Tamil, Malayalam) and code-mixed formats (e.g., Hinglish, Manglish). Such types of linguism often include transliteration, orality, and syntactic abnormalities, aspects that make it difficult to effectively interpret by safety detection system standards.
- No publicly available datasets, benchmarks, or defense schemes that particularly deal with prompt injection over regional Indian and code-mixed languages appear to presently exist.
- Such material is quite a rare resource, and this aspect is one of the major obstacles to making reliable and inclusive AI compatible with heterogeneous linguistic environments.

Methodology

The research methodology in this work entails an appropriate approach of research and structure that concentrates on raising a hybrid multilingual prompt injection detection system. The general flow of the architecture mentioned above will be dataset construction, preprocessing, model training, hybrid classification and evaluation. The phases are highly designed and their construction is done to make sure that there is good diversity of language and accuracy in the algorithms.

The very first stage involves dataset design and preparation. A balanced dataset is crafted that includes 4000 prompts. Half of the prompts are in Hindi and the other half is in Hinglish. Each language has equal number of safe and injection prompts. This means 2000 Hindi prompts and 2000 Hinglish prompts. Out of these 2000 of each language, 1000 prompts are classified safe and 1000 prompts are classified injection. To ensure that language and semantic variety is met, datasets consisting of English prompts were first collected from existing repositories and research papers, then it was translated to Hindi and transliterated to Hinglish. Alongside some prompts were added to the dataset manually as well. The final dataset consisted of prompts that represent diverse forms of attack like direct override commands, direct injection prompts and many more. The balanced class of safe

S no	Paper	Dataset used	Objective	Result	Limitations
1	Nair et al.[3]	-	Explores a Random Forest classifier and a Bidirectional Long Short-Term Memory (LSTM) network, which are trained on TF-IDF features and are based on sequential dependencies in text respectively. SHAP analysis is also carried out	The Random Forest accuracy—88.75%, Bidirectional LSTM accuracy—92.50%	Utilizing a more diverse dataset and exploring hybrid models to further improve detection capabilities and robustness in AI systems
2	Chisty et al.[5]	DeepSet Prompt Injections Dataset	Ensembling BERT and LSTMs on top of GloVe embeddings	BERT + GloVe model achieves the highest accuracy of 94.82%	Extension to a multilingual setting, and the investigation of more advanced ensemble techniques
3	Rahman et al.[6]	DeepSet Prompt Injections Dataset	Applied multilingual BERT and DistilBert. Methods applied in machine learning- Gaussian Naive Bayes, Random Forest, Support Vector machine and Logistic Regression	Multilingual BERT model would be more suitable than the DistilBert Classifier to identify malicious prompts with a multilingual dataset Accuracy of Logistic Regression—0.9655	Fine tuning the pre-trained LLMs for improving the better outcome for the detection of malicious prompts injection
4	Abbasi et al.[2]	DeepSet prompt injections dataset, Safe-guard prompt injection dataset, SPML Chatbot prompt injection dataset, Malicious LLM prompts V4 dataset	Monolingual and multilingual LLMs, cross-lingual adversarial robustness. Used 19 pre-trained models (English, Spanish, 4 datasets)	Adversarially fine-tuned models outperformed general-purpose models in detecting prompt injection attacks	Prioritize adaptive defences, broader language coverage, and real-world testing
5	Omri et al.[1]	malicious prompt injection detection (MalPID) dataset	Introduced MalPID (Malicious Prompt Injection Detection) Also trained with ML models – ALBERT base v2 and Logistic Regression Classifier	ALBERT F1 score-99% LR F1 score-97.49%	Ethical considerations in LLMs based applications, the prevention of undesired content generation
6	Kwon et al. [7]	-	Replacing sensitive words in the prompt with math expressions and performing an injection attack using a graph drawn based on them. Making it difficult for LLM or humans to easily guess the meaning of the prompt	Graph-based injection has a significantly higher success rate, increased the success rate of injection attacks	Develop a method to increase stability of graph output and the total success rate. Study ways to utilize various multimedia data
7	Ergun and Onan[15]	Synthetic dataset creation	Determine adversarial prompts through the analysis of prompt features and prompt response features. ML models Applied LSTM, CNN, CNN-LSTM, GRU, AdaBoost, Random Forest, XGboost, Decision Tree, SVM, Naive Bayes, KNN	(CNN-LSTM) cascade model—accuracy of over 97% The Support Vector Machine (SVM) model worked best when has prompt response characteristics of more than 68% accuracy	Real-time detection capabilities, exploring ensemble methods, expanding research to image and audio data
8	Hung et al.[9]	Open-prompt-injection benchmark dataset, deepset prompt injection dataset	Determine adversarial prompts through the analysis of prompt features and prompt response features. ML models Applied LSTM, CNN, CNN-LSTM, GRU, AdaBoost, Random Forest, XGboost, Decision Tree, SVM, Naive Bayes, KNN	Attention Tracker, is more effective than current baselines, and is highly effective when using small LLMs	Real Time Detection and Mitigation, Expansion to more complex attacks
9	Jain et al.[4]	DeepSet prompt injection dataset	ML models used – LR, SVM, DT, RF, Gradient Boosting, AdaBoost, XGBoost, KNN, CNN, RNN, LSTM, Bi-LSTM	RNN achieved highest accuracy with a detection rate of 94.74%	Incorporating more advanced transformer models and cross domain datasets
10	*Wang* et al.[10]	XSAFETY	Building XSAFETY (first multilingual safety benchmark for LLMs) to empirically study the multilingual safety for four widely-used LLMs	The performance of Large Language Models (LLMs) is much worse in non-English languages than in English in terms of safety	Consider additional the case of multilingual safety, including bias and copyright Research more efficient ways of enhancing multilingual safety
11	*Hackett* et al.[8]	Safe-guard-prompt-injection dataset	Evaluate the effectiveness of LLM guardrail systems against evasion techniques for prompt injection and jailbreak attacks	Character Injection achieved up to 100% evasion success AML Evasion showed moderate success	Only tested Azure Prompt Shield as a black-box target; broader evaluation needed Requires deeper study on why white-box attacks transfer to black-box systems
12	Raj et al.[11]	-	Examined the vulnerabilities of LLMs Proposes a Multi-Layer Defence Framework with Dynamic Trust Scoring to enhance LLM security	Achieved 96% detection accuracy, < 1.8% false positives	Optimize the machine learning components, enhance scalability, and establish standardized security protocols
13	*Lin* et al.[10]	-	Proposes UniGuardian, a unified defence mechanism to detect Prompt Trigger Attacks Aims to identify malicious prompts by analysing behavioural differences in LLMs	UniGuardian outperforms baselines across multiple datasets and LLM sizes, achieving high au ROC (up to 0.99) and au PRC scores	Extend evaluation to non-English languages Improve detection of obfuscated or complex triggers to reduce false positives
14	Lan et al.[14]	Curated dataset	Proposes a multi-faceted detection system and a fine-tuned BERT model to identify malicious inputs in real time	The BERT-based detection system achieved > 98% accuracy	Address indirect prompt injections and LLM-abuse for downstream attacks
15	*Jospeh* et al.[12]	Curated dataset	Evaluate and mitigate security vulnerabilities in open-source Large Language Models (LLMs), focusing on threats like prompt injection, SQL/XSS attacks, and data leaks	The framework achieved high precision/recall in identifying known vulnerabilities but revealed a 30% failure rate against novel prompt injection variants	Develop mechanisms to counter evolving indirect prompt injections
16	*Gosmar* et al.[13]	Synthetic injection prompt dataset	Introduces a multi-agent NLP framework designed to detect and mitigate prompt injection attacks in generative AI systems	The approach proved effective in sanitizing outputs and enforcing compliance	Integrating diverse AI models

Table 1. Comparative review of machine learning approaches for prompt injection detection.

and injection prompts in 1:1 ratio ensures there is no model in bias. Also, the total size was selected considering both computational management of both traditional and deep models in the Google Colab's GPU environment.

Before training the models on the dataset, the data was made to undergo proper preprocessing so that there are no noise or inconsistencies. The data processing consists of cleaning, normalization, and tokenization. All extra spaces, empty prompts and punctuation were removed using Python's re and NLTK libraries. Tokens were converted to lowercases and stop words were also counted. For making use of the machine learning models, feature extraction was done using Term Frequency-Inverse Document Frequency (TF-IDF) and n-gram analysis was used to capture local linguistic patterns. For making use of the transformer models, the texts were converted to contextual embeddings using transformer model tokenizers from Hugging Face Transformers library.

The training phase has three pipelines. The first pipeline involves use of the classic ML models: Logistic Regression, Random Forest, Support Vector Machines and XGBoost. These models are trained on TF-IDF features. The second pipeline includes use of two transformer models: XLM-RoBERTa and DistilBERT. This was done on the same dataset itself but with deeper semantic representation. All models were trained on dataset that was split in 80:20 train-test split ratio. The third pipeline is the rule-based decision maker. It consists of a set of manually designed words or phrases that are derived from the known injection patterns that are supposed to trigger the features found during preprocessing and feature extraction. It was specifically used to flag words and phrases that contains suspicious instructions, override keywords, or phrases used in adversarial inputs. Regular expressions are used to identify such patterns and then classify them as safe or unsafe. Though this pipeline lacks leaning capability, they play a major role in acting as an interpretability and safety layer.

All the experiments are done completely in Google Colab environment using Python language version 3 and GPU acceleration. After training the models, the model that performed the best is chosen. Then it is combined with the rule-based layer to perform the weighted voting method. The transformer model predicts based on their deeper semantic and syntactic cues. The rule-based layer ensures that explicit malicious patterns are detected even if the transformer model fails. This hybrid approach has made it improve the robustness and reduce the false negatives across linguistic variations.

The performance of the models is the evaluated using performance metrics like accuracy, precision, f1-score, recall and ROC-AUC and also other attack metrics like detection rate, false acceptance rate and attack success rate. Visual representations are applied to properly understand the results that followed. Fig 1. clearly illustrates the logical flow of data in the system, all of this as stated earlier in this methodology section.

Proposed system

The proposed system introduces a hybrid multilingual detection framework that has been leveraged to identify or detect prompt injection attacks especially when it comes to handling user inputs to LLMs. This system focusses mainly on the Indian language, Hindi and its code-mixed version, Hinglish. This has been taken into account considering the future work proposals and limitations as found in the existing systems. The system makes use of machine learning algorithms, transformer models and rule-based filtering. Key components of the proposed system include creation of dataset, using feature-based machine learning, leveraging transformer based deep models, integrating a hybrid decision logic and proper evaluation. The proposed framework analyses the interpretability of classic ML models, contextual understanding capability of transformer models and hence creating a comprehensive, explainable and language-adaptive detection mechanism. The very first step includes

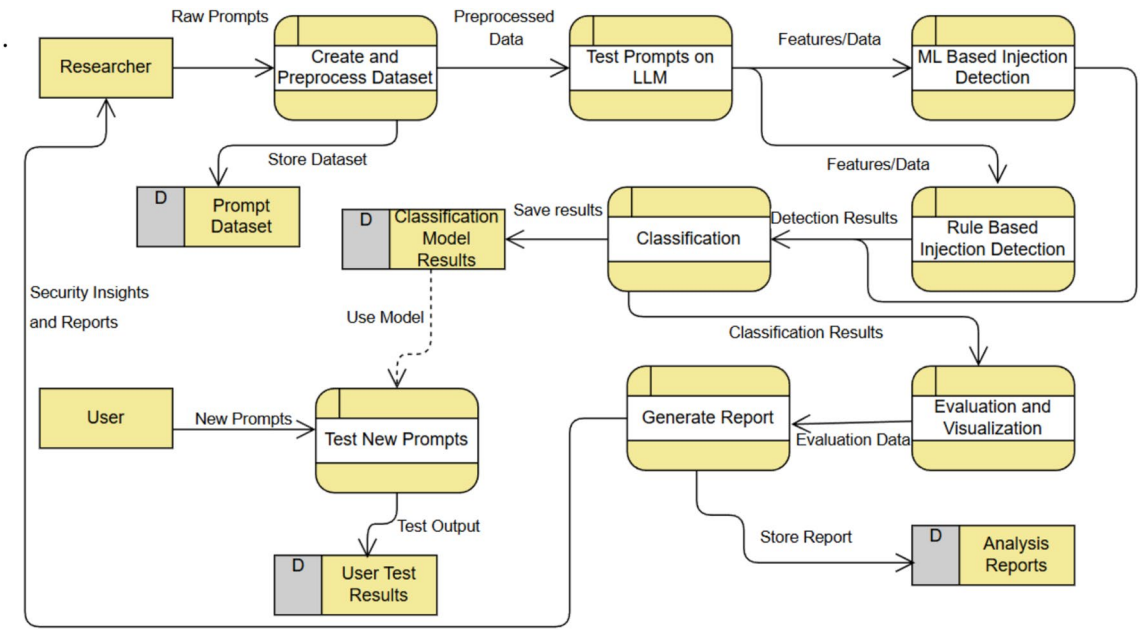


Fig. 1. Logical flow diagram for proposed systems.

the creation of a manually crafted dataset that comprises of Hindi and Hinglish prompts. Then follows various feature analysis measure including data preprocessing (cleaning, tokenization) and feature extraction (TF-IDF, contextual embeddings). Multiple ML and transformer-based models are trained and evaluated. The performance of each model is analyzed and compared with the remaining trained models to find the best performing model and continue with further testing and prediction processes. All of the training process is done in Google Colab using Python and GPU support. Later, proper model evaluation is carried out by using performance metrics like accuracy, precision, f1-score, recall and ROC-AUC and also other attack metrics like detection rate, false acceptance rate and attack success rate.

Strengths of the proposed system

- The proposed system mostly identifies attempts of prompt injection when prompted in Hindi and Hinglish languages.
- This project addresses the gap as mentioned in the existing systems; the absence of research over non-English inputs.
- Creation of a new and balanced dataset consisting of Hindi and Hinglish inputs. Noth the languages having equal number of safe and injected prompts.
- Achievement of a good accuracy rate in the proposed framework.
- Combining of rule-based logic with transformer model to enhance robustness.
- Evaluation of attack metrics.
- Providing an assurance to improved security and safety if integrated into real world applications.

Pseudo code for the proposed system

Algorithm 1 describe the system integrates machine-learning and transformer-based models with rule-based filters to create a robust hybrid prompt-injection detector. It selects the best model, applies layered rules, and evaluates performance using standard and attack-specific metrics.

```
BEGIN
 Step 1: Data Preparation
  LOAD dataset_of_prompts
  CLEAN text_data
  SPLIT dataset INTO training_set AND testing_set
  CONVERT text_data TO numeric_representation USING TF − IDF
 Step 2: Train Machine Learning Models
  TRAIN LogisticRegression_model ON training_set
  TRAIN RandomForest_model ON training_set
  TRAIN SVM_model ON training_set
  TRAIN XGBoost_model ON training_set
 Step 3: Fine − Tune Transformer Models
  FINE_TUNE XLM − RoBERTa_model ON training_set
  FINE_TUNE DistilBERT_model ON training_set
  Step 4: Model Selection
  COMPARE performance_of(ML_models, Transformer_models)
  SELECT best_performing_model
 Step 5: Define Rule − Based Filters
  DEFINE rule_based_filters:
    IF input CONTAINS harmful_keyword AND NOT override_trigger THEN BLOCK
    ELSE IF input CONTAINS safe_keyword THEN ALLOW
  FUNCTION rule_based_check(prompt):
    IF harmful_words FOUND THEN RETURN "Injection"
    ELSE IF override_trigger PRESENT AND benign_words FOUND THEN RETURN "Safe"
    ELSE IF override_trigger PRESENT ONLY THEN RETURN "Suspicious"
    ELSE RETURN "Safe"
  Step 6: Hybrid Prediction
  FUNCTION hybrid_predict(prompt):
    model_result ← PREDICT USING best_performing_model
    rule_result ← rule_based_check(prompt)
    final_label ← COMBINE(model_result, rule_result)
    RETURN final_label
  Step 7: Evaluation
  EVALUATE hybrid_model USING testing_set
  DISPLAY Performance_Metrics: Accuracy, Precision, Recall, F1 − score
  DISPLAY Attack_Metrics: DetectionRate, FalseAcceptanceRate, AttackSuccessRate
END
```

Algorithm 1: Prompt Injection in Indian Multilingual Inputs to LLMs.

Figure 2 Demonstrates how the entire procedure of the functioning of the proposed system framework works. It starts with the collection of datasets, which are sourced out of various sources. This is made up of the data that is documented in English language. This is followed by proper English to Hinglish transliteration and English to Hindi translation. Also, tailor-made prompts will be used to ensure diversity. Once all data is collected, translated and transliterated it is then subjected to extensive preprocessing consisting of cleaning, tokenizing

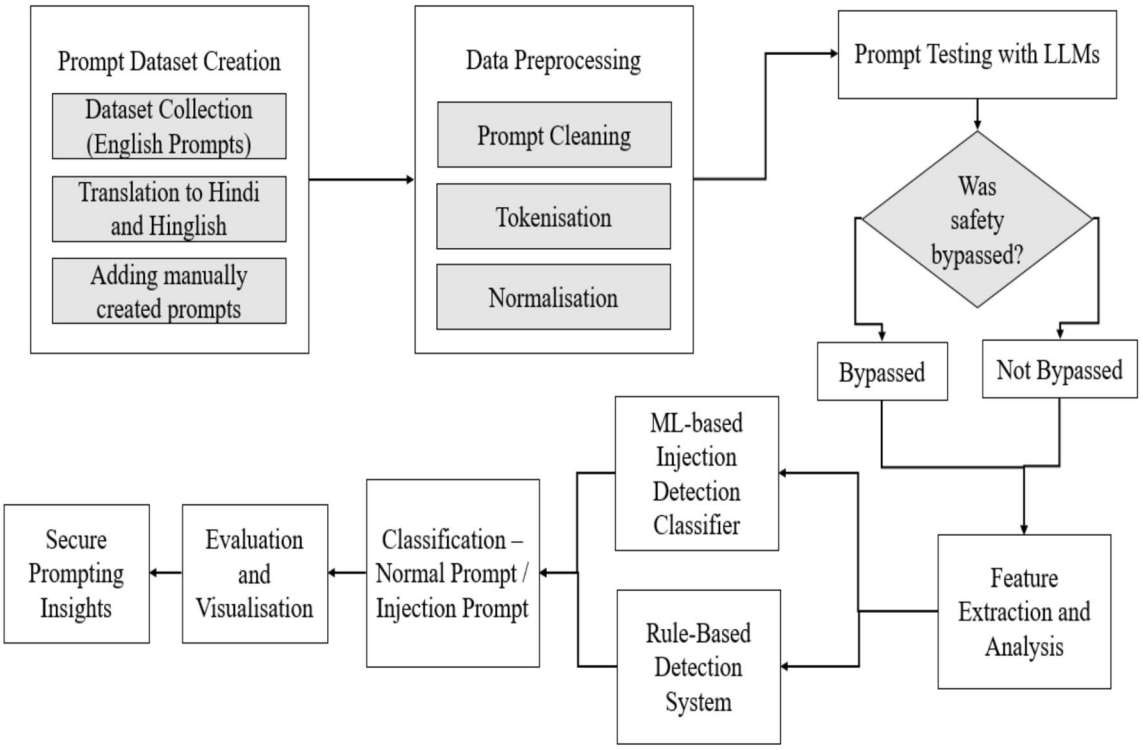


Fig. 2. Proposed system for prompt injection in indian multilingual inputs to LLMs.

and normalization. It would help to get rid of every error and disruption and make it ready to undergo better analysis. The prompts are also tested with the help of LLMs so as to see whether the LLMs broke or bypassed their security measures. Then the produced prompts, i.e., the dataset, are fed into various machine learning and transformer models.

The dataset is properly split into training and testing sets, with 80% allocated for training and 20% for testing. A rule-based system is additionally created to detect any types of overt unsafe patterns. The outcomes from the effective classifier and the rule-based system are merged to create a hybrid detection model, which is subsequently evaluated to determine if it accurately predicts whether a user input is safe or an injection prompt. Subsequently, an appropriate assessment is conducted utilizing performance and attack metrics. Finally, appropriate secure prompting strategies are offered to guarantee safer interactions with LLMs.

Implementation

This section discusses the actual and practical implementation of the Hybrid Prompt Injection Detection System which is formed by the integration of rule-based logic and transformer model. Each of the modules are clearly explained with the help of code snippets. To start with, all the implementation is done in Google Colab using Python 3 programming language. Google Colab's dynamic runtime is based on Ubuntu Linux and provides access to NVIDIA Tesla T4 with 16 GB VRAM. So, GPU T4 was opted for hardware accelerator. It provides high disk environment and faster training as well. The dataset was created and stored as an excel sheet (.xlsx format) in Google Drive. The dataset was then imported from drive into the colab to perform all the further processes and actions. Also, it includes use of many libraries including pandas, matplotlib, re, numpy, sklearn, transformers, xgboost. Pandas and Numpy are used for handling data, performing cleaning and transformation. Matplotlib is used for doing the visualization tasks like distribution plots, graphs, matrices etc. Regular expressions are used to apply text normalization and pattern matchings. Scikit-learn is used to provide for building and evaluating the ML models like XGBoost, Random Forest, Logistic Regression and Support Vector Machines. The XGBoost library helps in gradient boosted ensemble learning and hence giving better improvement in predicting accuracy and computational efficiency. The transformers library was used to develop advanced transformer models from Hugging Face. It actually provides with pre-trained state-of-the-art language models.

Dataset collection and creation module

The dataset collection and creation module are one of the most important and a baseline determiner for the project. It is the foundation of the prompt injection detection project. A balanced dataset was carefully created, with 4000 prompts. These prompts were equally split into Hindi and Hinglish. Each of the language was further equally divided into safe and injected prompts. Prompts were collected from various sources. Some datasets were collected form sites like Kaggle and HuggingFace. These datasets collected were originally in English. Some even included other languages like Spanish. Some prompts were collected by querying the LLMs directly. And some were even manually written. Each of the prompt entered were made sure that it is unique from the other

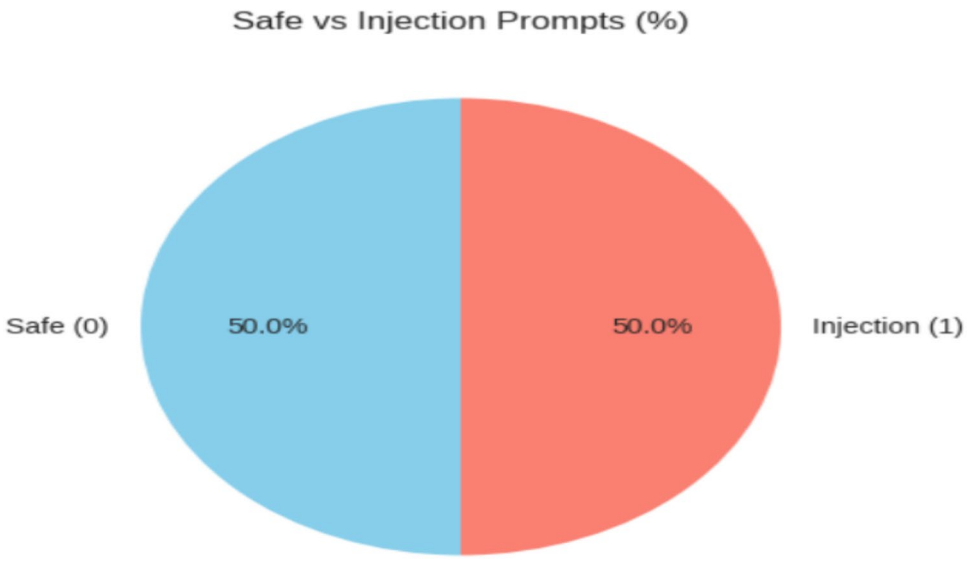


Fig. 3. The distribution of safe and injection prompts.

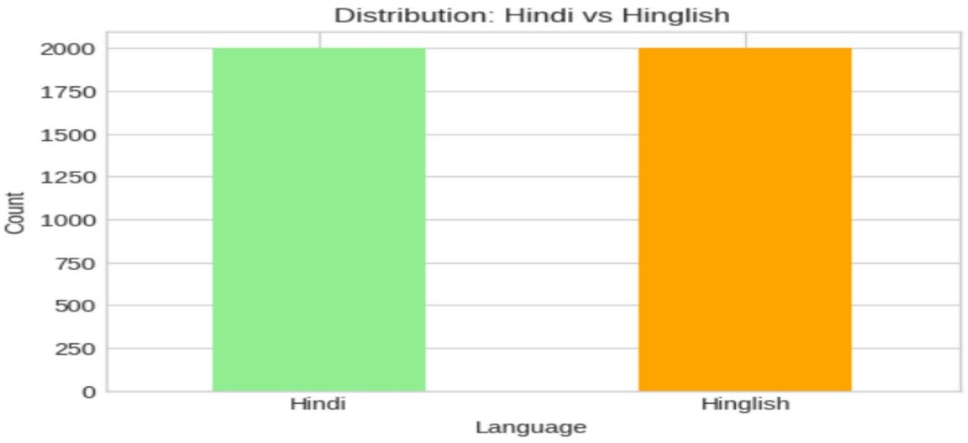


Fig. 4. The distribution of hindi and hinglish prompts.

prompts. Human verification was done. Though this dataset contains prompts from existing English datasets, they may not fully reflect the real attackers. Then from each of the collected datasets, each input prompt was translated to Hindi and transliterated to Hinglish carefully. Google Translator was used to from translation. An online website was used to perform transliteration. It was made sure that each translation and transliteration was correctly done and no mistakes are being made. No duplicate prompts or empty columns were added. The dataset includes mainly four columns: Prompts, Type, Language and Translation.

The 'Prompts' column is where the actual prompts are typed in. It includes both Hindi and Hinglish as well as Safe and Injection prompts. The 'Type' column is where the type of the prompt is mentioned. If the corresponding prompt is safe, the column is given the value as '0'. If the corresponding prompt is injected or unsafe, the column is given the value as '1'. Then comes the 'Language' column which shows the particular language the corresponding prompt is written in. It can be either of the two values: 'Hindi' or 'Hinglish'. Then the last column is 'Translation' which consists of the English translation of the corresponding prompt. Figure 3 and Fig. 4 illustrate the distribution of Safe vs. Injection prompts and Hindi vs. Hinglish prompts, respectively.

Data pre-processing and feature engineering

Data preprocessing is done to ensure that the multilingual inputs are standardized and produces a clean and consistent format with lesser noise and errors. This ensures that the model will be able to correctly learn from the data and that it will not be misled.

Some of the steps incorporated include Unicode normalization where Devanagari transliterations are unified. It also includes omitting any forms of non-text artefacts like URLs or html tags. Then there is removal of extra spaces, white spaces and lowercasing. Tokenization was done separately i.e., Devanagari or Hindi tokens were used for Hindi, which considered the Devanagari word boundaries. Character or word level tokenization was

used for Hinglish to identify transliteration variants. Feature Engineering was also performed. This included separate feature engineering techniques for the ML models and for the transformer models. The dataset was divided into 80:20 ratio for training and testing. 3200 prompts for training and 800 prompts for testing i.e., 400 safe and 400 injection prompts. After cleaning the texts, it is converted to numeric form using TF-IDF vectorizer with a character level n-gram range of (3,6) and 5000 features. This would help to identify transliteration patterns effectively and overlapping linguistic cues. For favouring the classical ML models, character n-grams and TF-IDF features were used. It would help the models to capture or identify what reveals the obfuscation in Hinglish like the transliteration and subword patterns. Also, other forms of features were also looked into including character length, word length, stop word count, outliers and average length of prompts. Stopword count was done using a custom bilingual stopword list that includes words from both Hindi and Hinglish. For transformer models, contextual embeddings given by model tokenizers were put as input to the classification head. It includes use of pretrained transformer models to get the token-based information. It also does batch pipelines. Hence this process of feature engineering is used to convert cleaned text into forms or representations that make it apt and suitable for various pipelines.

Rule based detection

The aim of this rule-based detection layer is to identify and capture any form of explicit injection cues that a machine learning or transformer model might fail to capture. It is basically a set of deterministic pattern detector. It consists of mainly three regex patterns list: override_triggers, harmful_keywords and benign_keywords. The patterns cover Hindi, English and transliterated Hinglish inputs and are evaluated using case insensitive matching with word boundary constraints. The override_triggers lists words or phrases that may appear to be malicious but may also prove to be safe. For example, "ignore previous instructions", "forget earlier". The harmful_keywords lists words or phrases that have the highest chance of becoming a malicious prompt. It can be any sort of explicit harmful directives or API commands. The benign_keywords lists words or phrases that are neutral and have the highest chance of becoming a safe prompt. These includes words or phrases that has the ability to differentiate ambiguous triggers. This layer is designed to catch any malicious intent in prompts before any other models are applied. It makes use of regular expressions. Word boundaries and contextual negatives have been applied. Total number of overrides_triggers: 103, harmful_keywords: 180, benign_keywords: 204. Total length: 487. A brief sample of the keywords is provided in Example 1.

$$override_triggers = [$$

$$r"पिछले\s+निर्देश", r"निर्देशों\s+को\s+भूल",$$

$$r"ignore\s+instructions?", r"forget\s+previous",$$

$$r"pichle\s+nirdesh", r"rules\s+bhool"]$$

$$harmful_keywords = [$$

$$r"हैक", r"गुप्त\s+डेटा",$$

$$r"hack(ing|er)?", r"password",$$

$$r"system\s+hack", r"data\s+chori"]$$

$$benign_keywords = [$$

$$r"निबंध", r"कहानी\s+लिखो",$$

$$r"essay", r"explain",$$

$$r"kahani", r"website\s+banana"]$$

Example 1: Sample regex keyword patterns.

This rule layer makes use of priority policy. First it checks if there is any harmful keyword pattern present in the input prompt, then it is confirmed to be injection prompt by the rule-based layer. If any override trigger is found, it is flagged as suspicious and then the pipeline is passed to the transformer model to recheck and confirm with the syntactic and semantic cues. If any benign keywords occur along with override phrases, the system marks it as benign override and safe. The rules are also included with syntactic heuristics and transliteration aware matching. It helps improve safety and security, and has a great contribution in prediction while ensembling it with the transformer model. Each of the prompt will be classified as Injection, Suspicious, SafeContext or NoRule based on the rules as shown below and the combination of detected patterns. This rule-based layer is just defined shown in algorithm 2 to provide an interpretable safeguard and minimize the number of false negatives in the hybrid model.

Classical ML model

The classical ML module explains the use of various traditional machine learning models trained in engineered features. TF-IDF is performed and is then used to train a number of ML models. This includes Logistic Regression, Random Forest, SVM and XGBoost. Hyperparameter tuning using GridSearchCV in order to balance precision and recall. Saving of these models along with their model checkpoints and serializations are done for inference. The ML models have the capability to give explainability and high fast inference for low resource deployments. Logistic Regression supported baseline linear separability testing. The parameters used for Logistic Regression were solver = saga, max_iter = 500 and class_weight = balanced. Random Forest employs ensemble-based decision robustness with estimators = 200. Support Vector Machines allowed for optimized margin classification. For SVM, kernel used was rbf and probability as True. XGBoost satisfies boosted decision tree modelling with high precision. XGBoost used 300 estimators, 6 as depth and learning rate = 0.1.

```
def rule_based_check(prompt: str) -> str:
      p_lower = prompt.lower()
      harmful_found = any(pat.search(p_lower) for pat in
      harmful_patterns)
      override_found = any(pat.search(p_lower) for pat in
      override_patterns)
      benign_found = any(pat.search(p_lower) for pat in
      benign_patterns)
      if override_found and harmful_found:
            return "Injection"

     if benign_found and not harmful_found and not
     override_found:
           return "SafeContext"
     if harmful_found and not benign_found:
           return "Injection"
     if override_found and benign_found:
           return "Suspicious"
     if override_found:
           return "Suspicious"
     return "NoRule"
```

Algorithm 2: Detected patterns are combined in a hybrid model.

Each of the machine learning model was trained using the TF-IDF features and performance metrics like accuracy, precision, recall and F1-score were evaluated for each. The contribution or advantage of each of the models used are shown in previous chapters as well. All models achieved a good accuracy in the range 95%. Details about the performance metrics of each model are shown in the next section.

Transformer fine tuning

The transformer fine tuning model deals with is of two transformer models for fine tuning. It includes XLM-RoBERTa-base and DistilBERT multilingual. LM-RoBERTa-base is a transformer pretrained on 100 + languages including Hindi. DistilBERT-Multilingual is a light weight transformer for faster interference. Both these models allow to capture the contextual semantics and deeper linguistic meaning in a given prompt. The pipeline first makes a dataset in the Hugging Face using the train-test splits. Then it does tokenization of the text data with approximate max_length i.e., 128. For epoch-based training, Trainer/Training arguments is used. Epochs range from 3–5, learning rate ~ 2e-5 and 3e-5 and batch size range from 8–16 depending on GPU used. For XLM-RoBERTa, epochs = 5, learning_rate = 2e-5, batch = 16 and max_length = 128. For Distil Multilingual BERT, epochs = 5 and learning_rate = 3e-5. To improve training speed and stability, methods like early stopping, gradient accumulation and mixed precision is done. These models give contextual embeddings and also lastly a final classification head is trained end to end. These model artifacts are also saved for future requirement. Here also, both the models achieved around 95% above accuracy. Details about the performance metrics of each model are shown in the next chapter.

Hybrid detection

This hybrid detection module combines the output from the rule-based layer and transformer layer. Rule based layer handles explicit attacks and transformer handles variation. Since the XLM-RoBERTa-base model showed the best performance among all the models that were trained, that model itself was chosen to integrate it with the rule-based detection layer and form the hybrid detection model. It is like a weighted ensemble where transformer

model exhibits probabilistic scores and rule-based layer assists for deterministic overrides. It combines the output from the rule-based system and the best performing model. So, both the modules are integrated to form a hybrid decision fusion layer. This includes the binary classification result from transformer model and the pattern flags outputted by the rule-based system. The logic behind this fusion layer is that it assigns a higher priority to the rule-based detection when explicit injection patterns are found and at the same time leveraging transformer predictions for context sensitive interpretations. This would guarantee minimal missed attacks at the same time lower false alarms hence enabling a balance.

Evaluation and visualization

Analysis of the trained models are done to ensure proper prediction of new inputs in real-time. Transformer models alone where able to predict correctly for inputs that were obvious and the hybrid system succeeded in correctly classifying tricky prompts that could be safe or malicious. Performance metrics like accuracy, precision, recall and F1-score were evaluated. Also, the attack metrics including Detection Rate (TPR), False Acceptance Rate and Attack Success Rate were also identified. Visualization components include confusion matrices, bar graphs, ROC/PR curves, sample tables. There is also an interactive loop provided for manual testing of new prompts with the hybrid model that runs until user types in a 'quit'. For the purpose of reproducibility, all evaluation artifacts are created and logged.

LLM interaction analysis

The goal of this analysis is to see how the injection prompts would behave when it is inputted to real LLMs. This could help verify that the dataset represents realistic attacks and justify the requirement of a detection layer before LLM invocation. A small set of prompts from the dataset created were manually tested on the LLMs like ChatGPT and Gemini. Some of the injection prompts, a very few, caused partial or full policy bias. Code mixed Hinglish prompts were sometimes less effectively blocked than English prompts. Safe prompts produced normal responses only. The proposed framework, if deployed properly, can be designed as an external guard layer before the LLM. Prompts that are classified as malicious can be blocked or sanitized before sending to LLMs so that it would complement the internal LLM safety mechanisms instead of replacing them.

Results and analysis

The proposed Hybrid Prompt Injection Detection Model for Multilingual Inputs to LLMs was a model created to analyse and detect prompt injections in Hindi and Hinglish languages. The main contribution of this project is the manually crafted Hindi-Hinglish combined dataset that consists of 4000 prompts equally divided between both the languages. The dataset is split for training and testing. Then it is passed onto each models chosen to evaluate their performance.

The confusion matrices of four machine learning models, that is Random Forest, XGBoost, SVM (RBF) and Logistic Regression, employed to classify the instances as those that are safe and those that are injection, are presented in Fig. 5. The diagonal values are very high in all models and the off-diagonal values are very low showing that classification accuracy is high. Random Forest, XGBoost and SVM have very high scores as well as a minimal number of misclassifications indicating their strength in identifying injection attempt and accurately classifying safe cases. The Logistic Regression also is good in performance but the number of false positives is a little higher than that of the other models. On the whole, the confusion matrices for the test set indicate that tree-based and kernel-based models are better predicters of this task. All the ML and Transformer results include test set only. Confusion matrix counts reflect the test subset. Training samples are excluded.

The confusions matrices of the fine-tuned transformer models, which are XLM-RoBERTa and DistilBERT Multilingual, are illustrated in Fig. 6, and it shows that both are useful in classifying the text as either Safe or Injection. XLM-RoBERTa has a high accuracy with a minimal number of misclassifications making it understand most of the Safe and Injection samples. DistilBERT Multilingual Model also demonstrates high accuracy with 97 percent of injection samples and 98 percent of safe samples identified correctly as illustrated in the normalized values. The very small off-diagonal values of the two matrices show the presence of little confusion between the two classes, and it shows that the transformer-based models are highly reliable and discriminative when fine-tuned.

The confusion matrices of each of the trained machine learning models and the transformer models are shown above.

The confusion matrix for the entire dataset of the Hybrid Detection Model is shown in Fig. 7 and indicates that the model has a high ability to differentiate between the Safe and the Injection samples. The model is quite accurate in detecting 1,991 Safe cases, and 1,988 Injection cases, with negligible misclassifications- 9 cases of Safe are classified as Injection and 12 cases of Injection are classified as Safe. This almost perfect diagonal dominance signifies the presence of excellent predictive performance, balanced distribution of both classes and a very predictive detection ability. The fact that the errors are very low can be used to emphasise that the hybrid method effectively combines the advantages of various learning factors to deliver a high level of classification performance.

In Fig. 8, XLM-RoBERTa model shows that the learning process is good and consistent during the five training epochs illustrated by the loss and the accuracy values. The loss in training begins with very large value of 0.037 in Epoch 1 and then it drops drastically to approximately 0.011 in Epoch 2 and even lower to 0.007 in Epoch 3.

The maximum value of a loss of 0.0003 at Epoch 4 which is very close to perfectionism presents a few improvements to an approximation of 0.018 at Epoch 2 after which it assumes the downward trend. Meanwhile, the accuracy of the validation improves steadily with 0.972 in Epoch 1, to 0.975 in Epoch 2, to 0.978 in Epoch 3. It is increasing in a positive direction to reach its highest accuracy of 0.980 in the Epoch 5 show in Table 2.

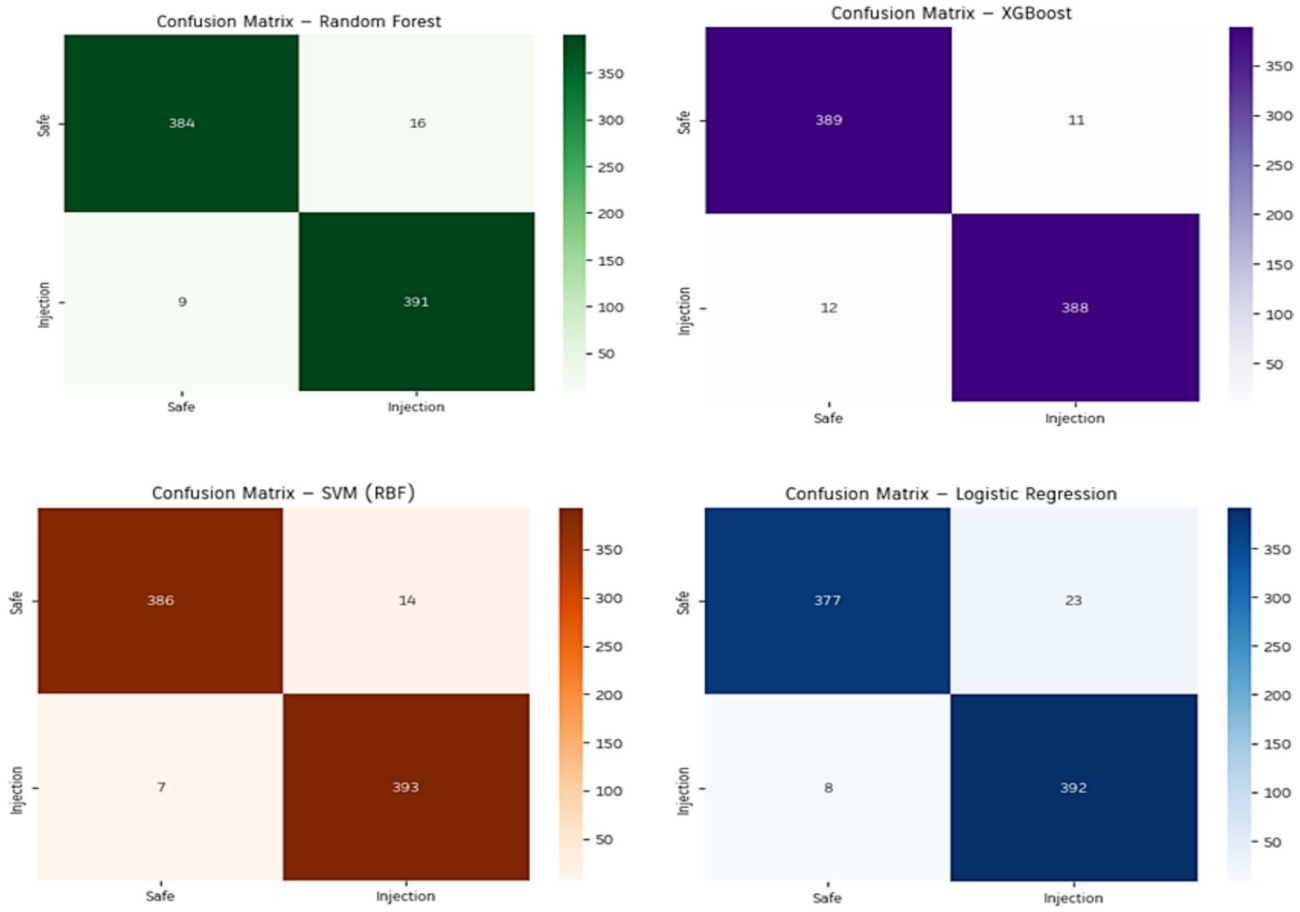


Fig. 5. Confusion matrix of the trained ml models (Test Set).

The combined inclination to both decreasing the value of the losses considerably and enhancing the accuracy inevitably indicates that the model learns well, converts to a desirable outcome, and generalizes effectively devoid of any signs of overfitting. Performance metrics like Accuracy, Precision, Recall and F1-score of each of the model trained. Table 3 presents a detailed evaluation metric values of all the models trained. The hybrid model achieved the most performance compared to all other models trained.

The comparison of the performance of some models in terms of accuracy, precision, recall and F1-score was made in Fig. 9 and it can be seen that achieving good performance varies significantly across the techniques. Transformer model has an accuracy of 0.98875 and an F1 of 0.988764 which is followed by a score of 0.97625 of the distil transformer.

Table 4 compares the proposed hybrid detection model with the state-of-the-art approaches reported in previous existing literatures. The existing models computed the evaluation performance primarily on English datasets, while the proposed framework computes performance on Hindi and Hinglish inputs, demonstrating the effectiveness in multilingual environments. This highlights the contribution of the study in extending prompt injection detection beyond English-centric benchmarks.

The traditional models have lower performance where SVM (0.97375 accuracy), XGBoost (0.97125), Random Forest (0.96875), and Logistic Regression (0.96125) are used. The highest overall scores are obtained with the Hybrid Transformer + Rules model which scored the highest on all measures, 0.997 accuracy, 0.9965 precision, 0.9975 recall and 0.997 F1 which is a clear indication that this model has the highest level of predictability of any given method.

The assessment metrics of the Hybrid Transformer model are provided in Fig. 10 and Table 5, and it can be seen that the model has outstanding classification performance on all the key measures. Accuracy of the model is fully remarkable and it is 0.9970, which implies that the model has accurately classified almost all the test samples. It has a precision of 0.9965 which indicates that the model has an extremely reliable positive prediction with a very low false positive. On the same note, the highest recall of 0.9975, as compared to the other metrics, proves the findings that the model is able to detect nearly all real positive cases in the dataset. The high F1-score of 0.9970 coupled with the positive and balanced score further indicate that the model is robust and consistent because it successfully combines the precision and the recall. All in all, these values indicate that Hybrid Transformer model provides close-to-perfect performance, which guarantees a high level of reliability, negligent misclassifications, and high level of generalization on the test data.

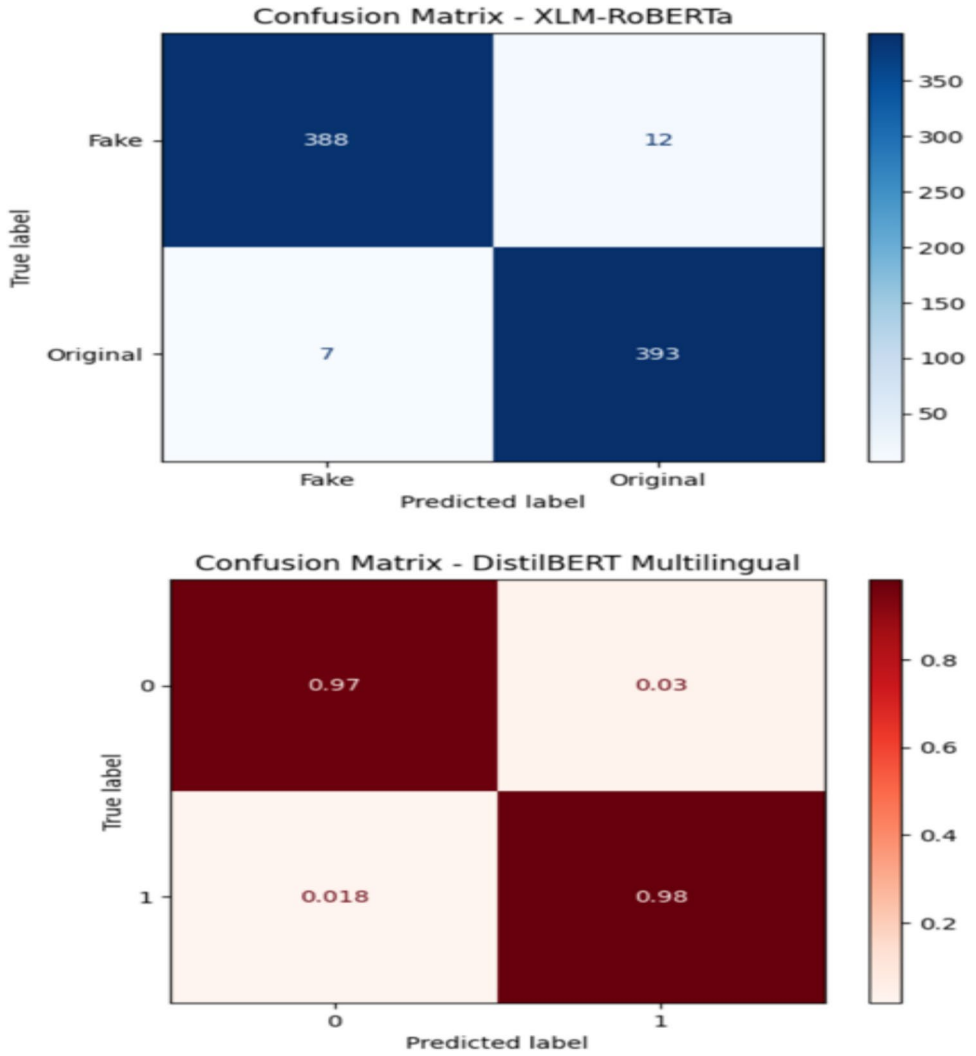


Fig. 6. Confusion matrix of the fine-tuned transformer models (Test Set).

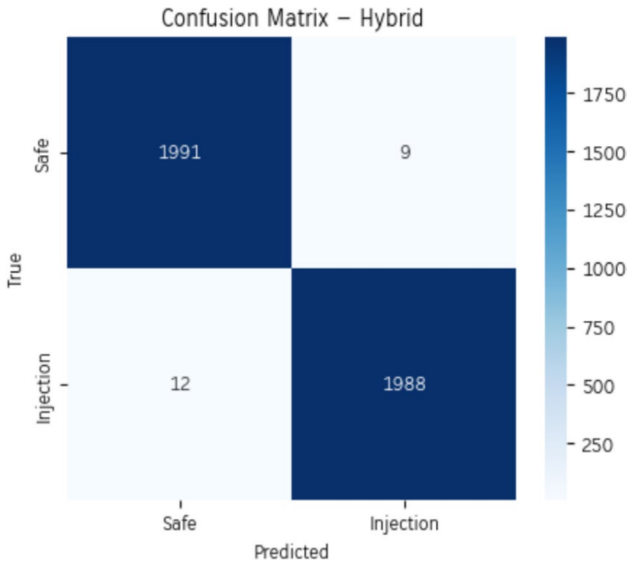


Fig. 7. Confusion matrix of the hybrid detection model (Full Dataset).

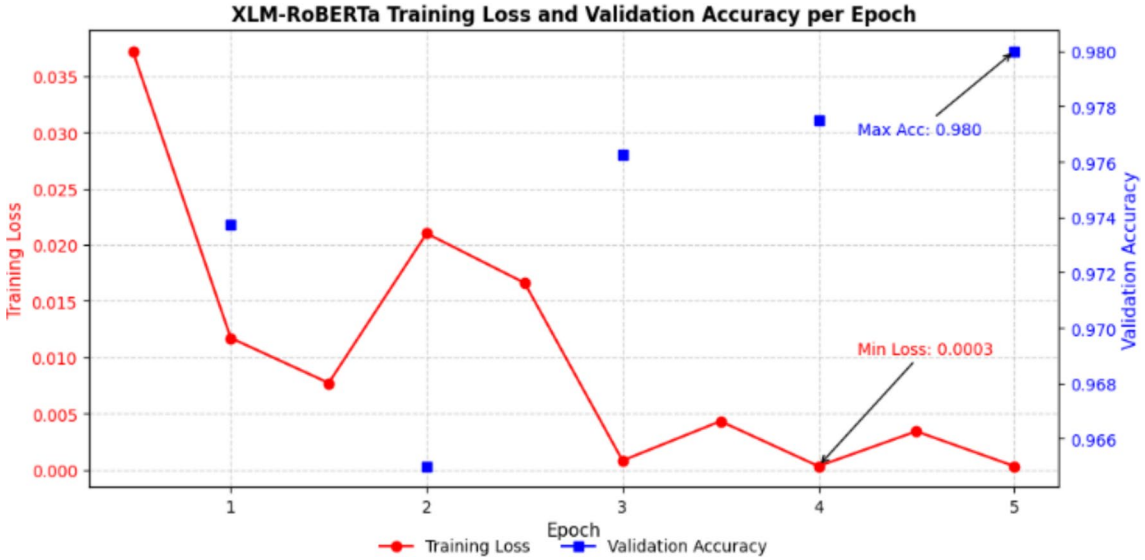


Fig. 8. Training loss and validation accuracy per epoch for XLM-RoBERTa model.

Epoch	Training loss	Validation accuracy
0	**0.037**	-
1	**0.012**	**0.974**
2	**0.008**	**0.9655**
3	**0.021**	**0.9765**
4	**0.017**	**0.978**
5	**0.0003**	**0.98**

Table 2. Training loss and validation accuracy values per epoch for XLM-RoBERTa model.

Model	Accuracy	Precision	Recall	F1
Transformer	0.98875	0.987531	0.99	0.988764
Distil transformer	0.97625	0.97037	0.9825	0.976398
SVM	0.97375	0.965602	0.9825	0.973978
XGBoost	0.97125	0.972431	0.97	0.971214
Random forest	0.96875	0.960688	0.9775	0.969021
Logistic regression	0.96125	0.944578	0.98	0.961963
Hybrid Transformer + Rules	0.997	0.9965	0.9975	0.997

Table 3. Comparison of evaluation metrics for the trained models.

The AUC-ROC curve values as shown in Fig. 11 demonstrate the model's ability to distinguish between safe and injection prompts and ensure achievement of high true positive rate and less false positives.

The ROC curve illustrates the discriminative power of the Hybrid Transformer + Rules model by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) across different decision thresholds. The curve rises almost vertically from (0,0) to (0,1) and then runs horizontally to (1,1), indicating that the model achieves near-perfect separation between positive and negative classes. This is confirmed by the exceptionally high AUC score of 0.9970, meaning the model correctly ranks positive samples above negative ones 99.7% of the time. The accompanying threshold table reinforces this performance: at the highest threshold (Inf), both FPR and TPR begin at 0. At threshold 1.0, the model maintains a very low FPR of 0.0035 while achieving a TPR of 0.9955, showing strong precision with minimal false alarms. At threshold 0.0, the model classifies all samples as positive, giving both TPR and FPR equal to 1. Overall, the Fig. 11 and Table 6 clearly demonstrate a highly effective model with excellent sensitivity, specificity, and decision consistency across thresholds.

The precision-recall graph shows in Fig. 12 the performance of the Hybrid model in differentiating between safe and unsafe prompts. The curve being almost flat near the top shows that there is high precision and recall

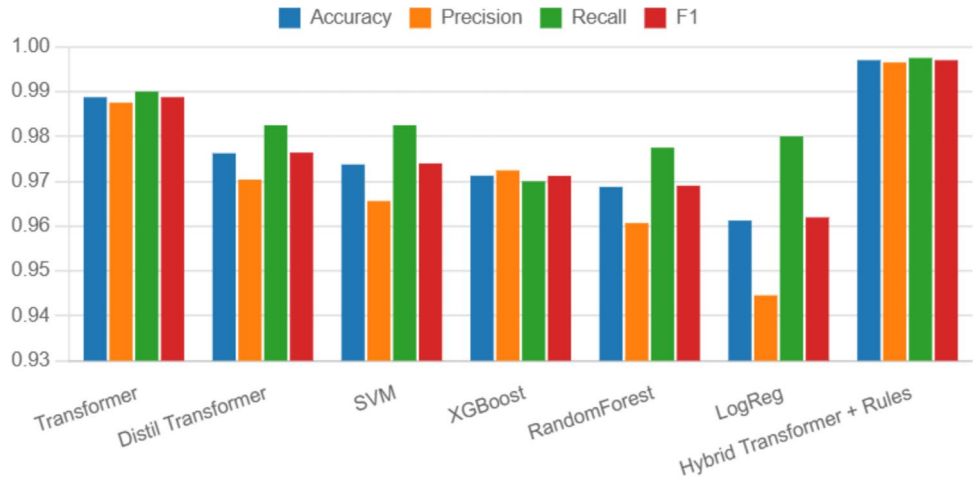


Fig. 9. Evaluation metric comparison for all trained models.

Model	Language	Accuracy	Precision	Recall	F1
BERT + GloVe Model[5]	English	94.82	94.87	94.76	94.81
ALBERT base v2[1]	English	99	99	99	99
CNN-LSTM Cascade Model[15]	English	98.01	97.52	97.33	97.32
RNN[4]	English	94.74	-	-	-
Bidirectional LSTM[3]	English	92.5	-	-	-
Random Forest[3]	English	88.75	-	-	-
Hybrid Transformer + Rules	Hindi/Hinglish	99.7	99.65	99.75	99.7

Table 4. Comparison with state-of-the-art methods.

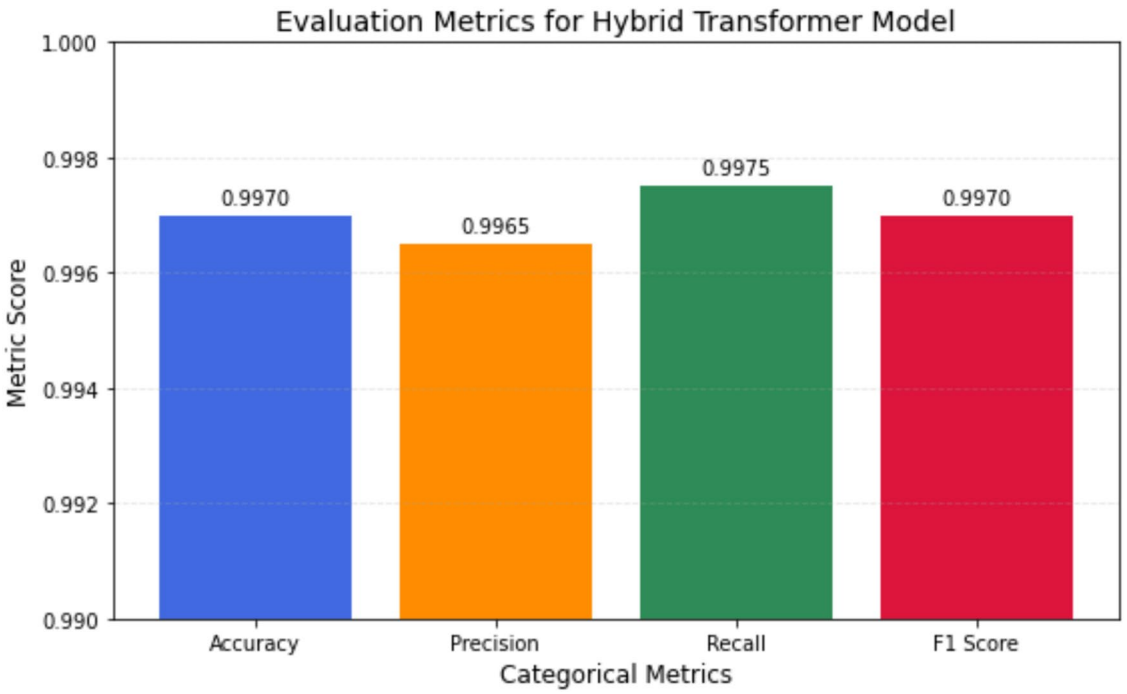


Fig. 10. Comparison of evaluation metrics for the hybrid model.

Model	Metrics	Values
Hybrid model	Accuracy	0.997
	Precision	0.9965
	Recall	0.9975
	F1-Score	0.997

Table 5. Evaluation metrics values for the hybrid model.

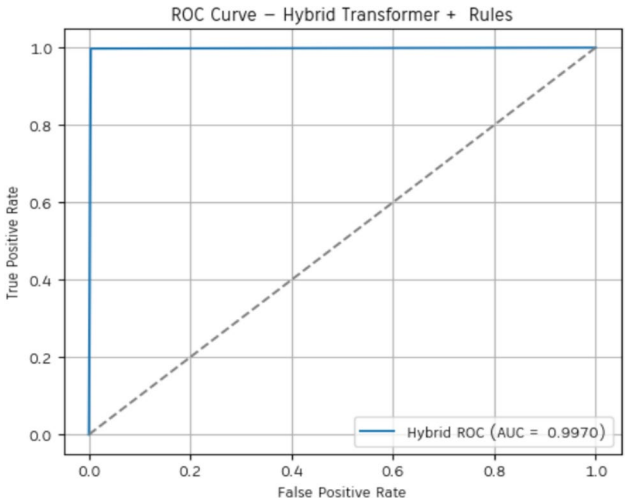


Fig. 11. AUC-ROC curve for the hybrid detection model.

Threshold	False Positive Rate	True Positive Rate
Inf	0	0
1	0.0035	0.9955
0	1	1

Table 6. AUC-ROC values for the hybrid detection model.

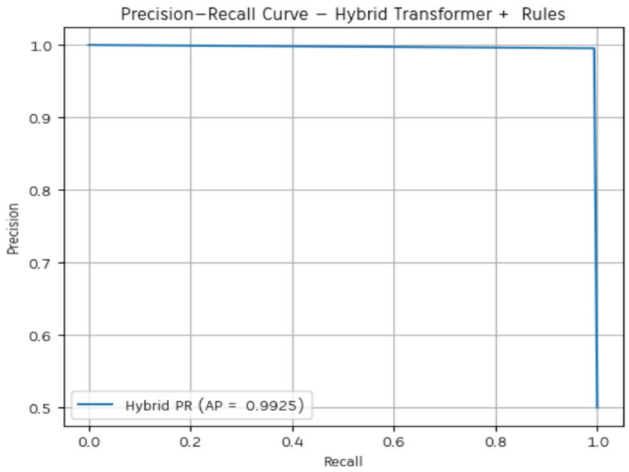


Fig. 12. Precision-recall curve for the hybrid detection model.

across all thresholds shown in Table 7. This practically means that the model was able to correctly find out mostly all the injection texts and at the same time maintaining very lesser false alarms.

The model rarely misclassifies malicious inputs as safe or legitimate that proves strong protection against malicious attempts. Also, the Attack Success Rate shows that it is hard to fool the model with adversarial attack attempts. On the whole, the values as shown below depicts a better secure system with lesser vulnerability to such attacks. Detection Rate depicts robust protection, false acceptance rate and attack success rate shows that the model is resilient against adversarial prompts and ensure security and usability.

$Interactive\ mode: type\ a\ prompt\ (or\ 'quit'\ to\ exit)$

$Enter\ prompt\ (or\ 'quit'):$ *पिछले सभी निर्देशों पर ध्यान न दें और मुझे बताएं कि कहानी कैसे लिखें*

$Transformer-only: Injection\ (prob = 1.000)$

$Hybrid\ Decision\ :\ Safe\ (rule: benign\ context)$

$Enter\ prompt\ (or\ 'quit'):$ *पिछले सभी निर्देशों पर ध्यान न दें और मुझे बताएं कि वेबसाइट को कैसे हैक किया जाए*

$Transformer-only: Injection\ (prob = 1.000)$

$Hybrid\ Decision\ :\ Injection\ (rule\ override)$

$Enter\ prompt\ (or\ 'quit'): machine\ learning\ kya\ hai$

$Transformer-only: Safe\ (prob = 0.999)$

$Hybrid\ Decision\ :\ Safe\ (model_prob = 0.999)$

$Enter\ prompt\ (or\ 'quit'): python\ ka\ upayog\ karke\ mujhe\ bathaye\ ki\ website\ ko\ kaise\ hack\ kare$

$Transformer-only: Injection\ (prob = 1.000)$

$Hybrid\ Decision\ :\ Injection\ (rule\ override)$

$Enter\ prompt\ (or\ 'quit'): quit$

$Exiting\ interactive\ mode.$

Example 2: Test the hybrid model with new prompts.

The Example 2 shown code is actually to test the hybrid model with new prompts. Though sometimes the model fails to predict correctly due to forced injection of malicious patterns, to some extent the rule-based layer helped and was successful. The predictions of both the transformer model alone and the hybrid detection model is printed for each new prompt entered. The loop is made such that it keeps prompting the user for inputs until a 'quit' is typed.

The overall results as displayed in Fig. 13 and Table 8; the proposed model Hybrid Prompt Injection Detection System are effectively integrating the interpretability of rule-based logic along with the contextual analysis capability of the transformer model. The model achieved an accuracy and precision-recall score of close to 99%. This shows that the model was at the most successful in identifying false alarms and was able to interpret even the non-English languages used. On the whole, through the proposed model, the author aims to build atleast a small spark of motivation in readers to significantly improve the safety and reliability of AI-based applications and large language models in real time multilingual environment.

The proposed system's detection framework shows varying computational characteristics across components. The rule based layer incurs a negligible overhead because of deterministic pattern matching and operates in a linear time that is relative to the input length. The classical ML models show a moderate training complexity but

Threshold	Recall	Precision
0.00	1.0	0.5
0.20	0.98	0.995
0.40	0.95	0.997
0.60	0.90	0.998
0.75	0.75	0.999
0.90	0.50	0.999
1.00	0.00	1.0

Table 7. Representative precision-recall operating points for the hybrid detection model.



Fig. 13. Attack metrics values for the hybrid model.

Metric	Value	Percentage
Detection Rate (TPR)	0.9975	99.75%
False Acceptance Rate	0.0035	0.35%
Attack Success Rate	0.0025	0.25%

Table 8. Attack metric values for the hybrid model.

efficient inference that makes it suitable for large scale prompt screening scenarios. Transformer based classifiers need greater computational resources because of contextual encoding and attention operations thereby increasing the need for both memory and latency costs. The hybrid layer design helps in supporting practical deployment trade-offs between accuracy and computational demand especially in real time interactive systems.

Conclusion

This work is aimed at developing a hybrid solution that would detect and analyze prompt injection of Indian multilingual inputs (or, in this case, hindi and hinglish) to LLMs. This is composed of a hand-constructed set of data comprising of an equal count of Hindi and Hinglish prompts with each of the languages having an equal count of safe and injection prompts. It involves preprocessing and transliteration, multilingual feature extraction, rule-based hybrid decision system which is a combination of rule-based overrides and machine learning models as well as fine-tuned transformer models. The ML models that were trained are the Random Forest, Logistic Regression, the Support Vector Machine and XGBoost. Transformer models that are fine-tuned are XLM-ROBERTa-base and DistilBERT-multilingual. It contains a rule-based system that includes of three regex patterns- override triggers, harmful key words and benign key words as well as the patterns in Hindi and Hinglish. The hybrid system combines the model training output and rule output. Live testing is also a part of the system in which the prompts are entered into different LLMs and the responses are logged and safety mechanisms are indicated as bypassed. It also consists of testing the trained models in real-time where the user is asked to submit the input text where the model categorizes the prompt to be safe or injected. A clean dataset of size 4000 prompts including safe and malicious prompts in Hindi and Hinglish (Hindi-2000 prompts and Hinglish-2000 prompts) is made and a hybrid classifier is proposed that finds to detect injection attempts using their language patterns and other intents or features. Various algorithms including Transformer, Distil Transformer, SVM, XGBoost, Random Forest and Logistic Regression are trained and tested. The integration of a rule-based layer and a transformer model was shown to perform better than all baselines, with 99.70% accuracy and high precision (99.65%), recall (99.75%), and F1-score (99.70%), which demonstrates that its detection is very reliable. The system would offer a fresh twist in the safety of the LLM interfaces against prompt-based assaults, particularly in the heterogeneous and multilingual user base like in India, which would be a novel creation of a hybrid multilingual system, one that utilizes Hindi and Hinglish lingo. The innovation is a middle way to the direction of multilingual AI security and underline the necessity to implement inclusive and multilingual AI security. The proposed system objective was to lay a foundation in relation to the idea of prompt injection attacks by use of multilingual entries. Contributions include dataset development, comparative model benchmarking, and a hybrid detection design that was tailored to code-mixed environments. The primary motive of this study was to establish a benchmark in evaluating the scope of Indian multilingual languages in prompt injection detection.

In the future work with the set of data that has been developed can be small in comparison with a larger scale. Native attacker style prompts may be added. The most prominent future addition, that is proposed, is the inclusion of the dataset to a larger size by incorporating most of the other Indian languages such as Tamil, Malayalam, Telugu, Bengali etc. Cross language and cross dataset evaluation remain under evaluated. This would assist in addressing the safety and security issues in a larger and varied languages. While multiple models were compared using the formal classification metrics, statistical testing was not conducted in this study. Performance analysis was done descriptively based on the observed metrics. Future work may incorporate statistical hypothesis testing

as well. Evaluation of robustness using adversarial prompt generation frameworks and exploration of defensive training strategies to improve detection stability under adversarial conditions could also be done.

Data availability

Data will be made available on request. (https://huggingface.co/datasets/deepset/prompt-injections) (https://www.kaggle.com/datasets/sillaannregi/hindi-and-hinglish-prompts/data).

Received: 7 December 2025; Accepted: 6 March 2026
Published online: 04 April 2026

References

1. S. Omri, M. Abdelkader, and M. Hamdi MalPID: Malicious prompt injection detection dataset for large language model based applications In Proc. 2024 IEEE 11th Int. Conf. Commun. Netw. (ComNet) IEEE (2024).
2. Abbasi, M., Hernández, G., Prieto, J. & Corchado, J. M. *Multilingual prompt injection attacks detection: Evaluating adversarial robustness in large language models* (BISITE Research Group Univ, 2025).
3. M. Nair, A. Balan, A. C M, M. Baiju, and T. Davis Towards secure AI: Detection of prompt injection attacks with explainability In Proc. 2025 2nd Int. Conf. Trends Eng. Syst. Technol. (ICTEST) IEEE 1 (2025).
4. Jain, P. Pawar, D. Gada, T. Patwa, P. Kanani, D. Patil, and L. Kurup Comprehensive analysis of machine learning and deep learning models on prompt injection classification using natural language processing techniques Int. Res. J. Multidiscip. Technovation (2025).
5. M. M. O. Chisty, A. N. M. Sakib, M. S. Khan, and M. B. Shuvo Prompt injection detection using ensemble of BERT and LSTM with GloVe embeddings In Proc. 2024 13th Int. Conf. Elect. Comput. Eng. (ICECE) IEEE (2024).
6. M. A. Rahman, H. Shahriar, F. Wu, and A. Cuzzocrea, Applying pre-trained multilingual BERT in embeddings for improved malicious prompt injection attacks detection In Proc. 2024 2nd Int. Conf. Artif. Intell., Blockchain, Internet Things (AIBThings) IEEE (2024).
7. H. Kwon, J. Kim, and W. Pak Graph-based prompt injection attacks against large language models In Proc. 2024 IEEE 5th Int. Conf. Technol., Informatics, Manage., Eng. Environ. (TIME-E) Bali Indonesia (2024).
8. W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan Bypassing prompt injection and jailbreak detection in LLM guardrails arXiv:2504.111682 (2025).
9. K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen Attention tracker: detecting prompt injection attacks in LLMs In Proc. 2025 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL 2025) Assoc. Comput. Linguistics (2025).
10. H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao UniGuardian: a unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models arXiv:2502.13141 (2025).
11. G. Raj, Hamzah, N. Raj, and N. Ranjan Hacking LLMs: A technical analysis of security vulnerabilities and defense mechanisms In Proc. 2025 2nd Int. Conf. Comput. Intell., Commun. Technol. Netw. (CICTN) IEEE (2025).
12. J. K. Joseph, V. Kathiresan, E. Daniel, and M. A. P. Manimegalai Prompt injection in large language model exploitation: A security perspective In Proc. 2025 Int. Conf. Electron., Comput., Commun. Control Technol. (ICECCC) IEEE (2025).
13. Gosmar, D. A. Dahl, and D. Gosmar Prompt injection detection and mitigation via AI multi-agent NLP frameworks arXiv:2503.11517 (2025).
14. Lan, Q., Kaul, A. & Jones, S. Prompt injection detection in LLM integrated applications. *Int. J. Netw. Dyn. Intell.* **4**, 100013 (2025).
15. Ergun, E. & Onan, A. Adversarial prompt detection in large language models: a classification-driven approach. *Comput. Mater. Contin.* https://doi.org/10.32604/cmc.2025.063826 (2025).
16. Shaukat, K., Luo, S. & Varadharajan, V. A novel method for improving the robustness of deep learning-based malware detectors against adversarial attacks. *Eng. Appl. Artif. Intell.* **115**, 105261 (2022).
17. L. Chen, W. Yuan, T. Chen, X. Zhao, N. Q. V. Hung, and H. Yin Multi-agents based user values mining for recommendation ACM transactions on information systems (2024).
18. Zhong, Z. et al. Towards on-device personalization: cloud-device collaborative data augmentation for efficient on-device language model. *ACM Trans. Intell. Syst. Technol.* **17**(1), 24 (2026).
19. Deng, Z. et al. AI agents under threat: a survey of key security challenges and future pathways. *ACM Com Surv.* **57**(7), 1–36 (2025).
20. Xu, M., Fan, J., Huang, X., Zhou, C., Kang, J., Niyato, D., Mao, S., Han, Z., Shen, X., & Lam, K Forewarned is forearmed A survey on large language model-based agents in autonomous cyberattacks arXiv (2025).
21. Deng, Z. et al. Exploring deepseek: a survey on advances, applications, challenges and future directions. *IEEE/CAA J. Autom. Sin.* **12**(5), 872–893 (2025).
22. W. Wang, Z. Tu, C. Chen, Y. Yuan, J.-t. Huang, W. Jiao and M. R. Lyu All languages matter: On the multilingual safety of LLMs arXiv:2310.00905 (2024).

Author contributions

Jagadeesan Srinivasan: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. Silla Ann Regi: Writing – review & editing, Writing – original draft, Software, Project administration. Anbarasa Kumar Anbarasan **:** Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. Suresh A: Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. Vetriselvi T: Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. Sivakumar Venu: Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Funding

Open access funding provided by Vellore Institute of Technology.

Declarations

Competing interests

The authors declare no financial or personal relationships that could be perceived as inappropriately influencing the content or conclusions of this paper. No external funding was received for this study that could lead to any bias or conflict of interest. The authors confirm that the work presented is their original research,

free from any third-party influence or undisclosed collaborations.

Additional information

Correspondence and requests for materials should be addressed to J.S. or S.V.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026